Below is a **complete detailed explanation** + a **ready-to-run project named java_multithreading_project** + a **comparison table** showing:

- How multithreading evolved in Java **from Java 1 to Java 21**
- How **Executor Framework** works
- Difference between **traditional multithreading** vs **Executor Framework**
- Modern **Virtual Threads (Java 21)** examples

---

1) What is Multithreading in Java?

✅ Meaning

Multithreading means executing multiple tasks (threads) concurrently within the same process (JVM).
Each thread has:

- its own stack
- its own program counter
- shared heap memory with other threads

✅ Why multithreading?

- better performance (parallel execution)
- responsiveness
- resource utilization
- scalable concurrency

---

2) How Threads Work Internally in JVM

Thread Lifecycle

A thread goes through:

1. **NEW** (created)
2. **RUNNABLE** (ready to run)
3. **RUNNING** (executing)
4. **WAITING / TIMED_WAITING** (sleep/join/wait)
5. **BLOCKED** (waiting for lock)
6. **TERMINATED** (completed)

---

3) Java Multithreading Evolution (Java 1 → Java 21)

✅ **Important improvements by version**

- **Java 1.0 / 1.1**: Thread, Runnable, synchronized, wait/notify
- **Java 5**: java.util.concurrent (Executor Framework, Locks, Atomic)
- **Java 7**: ForkJoinPool, Phaser
- **Java 8**: CompletableFuture, parallel streams
- **Java 9**: Flow API (reactive streams)
- **Java 19/20/21**: Virtual threads (Project Loom)
- **Java 21**: Virtual threads stable (production-ready)

---

4) Traditional Multithreading vs Executor Framework (Core Difference)

Traditional multithreading:
✅ You create threads manually
❌ No pooling
❌ Hard error handling
❌ Hard scaling
❌ Hard scheduling

Executor Framework:
✅ You submit tasks
✅ Thread pool reuses threads
✅ Future result support
✅ Scalable + optimized
✅ Scheduling supported

---

5) Comparison Table (Traditional Threads vs Executor Framework vs Virtual Threads)

Feature	**Traditional Multithreading (Thread)**	**Executor Framework**	**Virtual Threads (Java 21)**
Who manages threads?	Developer	ExecutorService	JVM
Thread creation cost	High	Moderate (pool reuse)	Very low
Scalability	Poor for high concurrency	Good	Best (millions possible)
Resource usage	Heavy	Medium	Light
Scheduling	Manual	Built-in (ScheduledExecutorService)	JVM managed

Error handling	Hard	Better (Future/Callable)	Simple + scalable
Best for	Small apps	Enterprise apps	High IO apps / microservices
Suitable for blocking IO	Expensive	Still expensive	Perfect
Debugging	Easy	Medium	Needs understanding

---

6) ✅ Create Project: java_multithreading_project

✅ Directory Structure

```
java_multithreading_project/
│
├── pom.xml
├── README.md
└── src/
    └── main/
        └── java/
            └── com/
                └── multithreading/
                    ├── App.java
                    ├── traditional/
                    │   ├── ThreadExample.java
                    │   ├── RunnableExample.java
                    │   └── SynchronizationExample.java
                    ├── executor/
                    │   ├── ExecutorFixedPoolExample.java
                    │   ├── CallableFutureExample.java
                    │   ├── ScheduledExecutorExample.java
                    │   └── CompletableFutureExample.java
                    └── virtualthreads/
                        ├── VirtualThreadExample.java
                        └── VirtualThreadExecutorExample.java
```

---

7) Full Code for Project

✅ pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.multithreading</groupId>
  <artifactId>java_multithreading_project</artifactId>
  <version>1.0</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

</project>
```

✅ App.java

```
package com.multithreading;

import com.multithreading.executor.*;
import com.multithreading.traditional.*;
import com.multithreading.virtualthreads.*;

public class App {

  public static void main(String[] args) throws Exception {

    System.out.println("===== Traditional Multithreading =====");
    ThreadExample.run();
    RunnableExample.run();
    SynchronizationExample.run();

    System.out.println("\n===== Executor Framework =====");
    ExecutorFixedPoolExample.run();
    CallableFutureExample.run();
    ScheduledExecutorExample.run();
    CompletableFutureExample.run();

    System.out.println("\n===== Virtual Threads (Java 21) =====");
    VirtualThreadExample.run();
    VirtualThreadExecutorExample.run();
  }
}
```

✅ Traditional Multithreading Examples

✅ **ThreadExample.java**

```
package com.multithreading.traditional;

public class ThreadExample {

   static class MyThread extends Thread {
      @Override
      public void run() {
         System.out.println(Thread.currentThread().getName() + " running task...");
      }
   }

   public static void run() {
      Thread t1 = new MyThread();
      Thread t2 = new MyThread();

      t1.start();
      t2.start();
   }
}
```

✅ **RunnableExample.java**

```
package com.multithreading.traditional;

public class RunnableExample {

   public static void run() {
      Runnable task = () -> System.out.println(Thread.currentThread().getName() + " executing
Runnable");

      Thread t1 = new Thread(task, "Runnable-Thread-1");
      Thread t2 = new Thread(task, "Runnable-Thread-2");

      t1.start();
      t2.start();
   }
}
```

✅ **SynchronizationExample.java**

```
package com.multithreading.traditional;

public class SynchronizationExample {

   static class Counter {
```

```
        private int count = 0;

        public synchronized void increment() {
            count++;
        }

        public int getCount() {
            return count;
        }
    }

    public static void run() throws InterruptedException {
        Counter counter = new Counter();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final counter value = " + counter.getCount());
    }
}
```

✅ Executor Framework Examples

✅ ExecutorFixedPoolExample.java

```
package com.multithreading.executor;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorFixedPoolExample {

    public static void run() {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 5; i++) {
            int taskId = i;
            executor.submit(() -> {
```

```
            System.out.println("Task " + taskId + " executed by " +
Thread.currentThread().getName());
        });
    }

    executor.shutdown();
  }
}
```

✅ **CallableFutureExample.java**

```
package com.multithreading.executor;

import java.util.concurrent.*;

public class CallableFutureExample {

  public static void run() throws Exception {
    ExecutorService executor = Executors.newSingleThreadExecutor();

    Callable<String> callTask = () -> {
      Thread.sleep(500);
      return "Result from " + Thread.currentThread().getName();
    };

    Future<String> future = executor.submit(callTask);

    System.out.println("Waiting for result...");
    System.out.println("Future result = " + future.get());

    executor.shutdown();
  }
}
```

✅ **ScheduledExecutorExample.java**

```
package com.multithreading.executor;

import java.util.concurrent.*;

public class ScheduledExecutorExample {

  public static void run() throws InterruptedException {
    ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);

    scheduler.schedule(() -> {
```

```
        System.out.println("Scheduled task executed after 2 seconds by " +
Thread.currentThread().getName());
    }, 2, TimeUnit.SECONDS);

    scheduler.shutdown();
    scheduler.awaitTermination(5, TimeUnit.SECONDS);
  }
}
```

✅ CompletableFutureExample.java

```
package com.multithreading.executor;

import java.util.concurrent.*;

public class CompletableFutureExample {

  public static void run() throws Exception {

    CompletableFuture<Integer> cf = CompletableFuture
        .supplyAsync(() -> {
          System.out.println("supplyAsync in " + Thread.currentThread().getName());
          return 10;
        })
        .thenApply(value -> {
          System.out.println("thenApply in " + Thread.currentThread().getName());
          return value * 5;
        })
        .thenApply(result -> result + 100);

    System.out.println("CompletableFuture final result = " + cf.get());
  }
}
```

✅ Virtual Threads (Java 21)

✅ VirtualThreadExample.java

```
package com.multithreading.virtualthreads;

public class VirtualThreadExample {

  public static void run() throws InterruptedException {

    Thread vThread = Thread.startVirtualThread(() -> {
```

```
        System.out.println("Virtual thread running: " + Thread.currentThread());
    });

    vThread.join();
  }
}
```

✅ **VirtualThreadExecutorExample.java**

```
package com.multithreading.virtualthreads;

import java.util.concurrent.*;

public class VirtualThreadExecutorExample {

  public static void run() throws Exception {

    try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {

      for (int i = 1; i <= 5; i++) {
        int taskId = i;
        executor.submit(() -> {
          System.out.println("Virtual Task " + taskId + " running on " + Thread.currentThread());
          Thread.sleep(300);
          return taskId;
        });
      }
    }
  }
}
```

---

8) README.md (Run Instructions)

✅ **README.md**

```
# java_multithreading_project

## Requirements
- Java 21
- Maven

## Run
mvn clean package
mvn exec:java -Dexec.mainClass="com.multithreading.App"
```

---

9) Important Real-Time Usage (When to use what?)

Need	Best Option
Small demo or learning	Thread / Runnable
Enterprise concurrency	ExecutorService
Scheduled tasks / Cron like jobs	ScheduledExecutorService
Async pipeline processing	CompletableFuture
Massive IO (microservices, socket, db calls)	Virtual Threads (Java 21)
CPU Intensive parallelism	ForkJoinPool / Parallel Streams

If you want, I can also give you:
✅ a **Zip structure output** for this project
✅ additional modules like **Locks, ReentrantLock, ReadWriteLock, Semaphore, CountDownLatch, CyclicBarrier, Phaser** with real-time examples
✅ Thread dump debugging commands and profiling tips for production systems

Below is a **detailed explanation of CompletableFutureExample** with a **complete working code example**, and everything is explained **in the form of tables** (as you asked).

---

1) What is CompletableFuture?

✅ CompletableFuture is an **asynchronous programming API** introduced in **Java 8**, used to:

- run tasks asynchronously (non-blocking style)
- chain tasks (pipeline)
- combine multiple async tasks
- handle exceptions cleanly

It is part of:
java.util.concurrent

---

2) CompletableFutureExample (Original Pipeline)

✅ Code Example (same concept as earlier)

CompletableFuture<Integer> cf = CompletableFuture

```
    .supplyAsync(() -> {
        System.out.println("supplyAsync in " + Thread.currentThread().getName());
        return 10;
    })
    .thenApply(value -> {
        System.out.println("thenApply in " + Thread.currentThread().getName());
        return value * 5;
    })
    .thenApply(result -> result + 100);

System.out.println("CompletableFuture final result = " + cf.get());
```

---

3) CompletableFutureExample Explanation (Step-by-step Table)

Step	Method	Type	What it does	Thread behavior	Output/result
1	supplyAsync()	Async Start	Starts an async task and returns a value	Runs on **ForkJoinPool.commonPool** (default) thread	returns 10
2	thenApply()	Sync Transform	Takes output of previous stage and transforms it	Runs on **same async pipeline thread**	10 * 5 = 50
3	thenApply()	Sync Transform	Another transformation on previous stage result	same pipeline thread	50 + 100 = 150
4	get()	Blocking	Waits and fetches the final result	blocks main thread	prints 150

---

4) Real Time Meaning (What Happens)

Action	Meaning in Real-life
supplyAsync()	Start background work (DB/API/file read)
first thenApply()	Process raw result (convert / validate / filter)

second thenApply()	Map into final business output
get()	Wait only at the end (if needed), for final output

---

5) Improved Example (Full Program)

✅ This example clearly shows:

- async execution
- chaining
- final result
- thread names

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureExample {

   public static void main(String[] args) throws Exception {

      System.out.println("Main thread: " + Thread.currentThread().getName());

      CompletableFuture<Integer> cf = CompletableFuture
            .supplyAsync(() -> {
               System.out.println("Stage-1 supplyAsync running in: " +
Thread.currentThread().getName());
               return 10;
            })
            .thenApply(value -> {
               System.out.println("Stage-2 thenApply running in: " +
Thread.currentThread().getName());
               return value * 5;
            })
            .thenApply(result -> {
               System.out.println("Stage-3 thenApply running in: " +
Thread.currentThread().getName());
               return result + 100;
            });

      System.out.println("Final result = " + cf.get()); // blocking
   }
}
```

---

6) Output Understanding Table

Example output:

Main thread: main
Stage-1 supplyAsync running in: ForkJoinPool.commonPool-worker-1
Stage-2 thenApply running in: ForkJoinPool.commonPool-worker-1
Stage-3 thenApply running in: ForkJoinPool.commonPool-worker-1
Final result = 150

Line	What it means
Main thread: main	main thread created the pipeline
Stage 1–3 worker thread	async pipeline is executed by ForkJoinPool thread
Final result = 150	main thread waited at .get() to print final

---

7) Important Methods in CompletableFuture (Table)

Method	Purpose	Returns	Blocking?
runAsync()	async task but no return value	CompletableFuture<Void>	✗
supplyAsync()	async task returning value	CompletableFuture<T>	✗
thenApply()	transform result	CompletableFuture<R>	✗
thenAccept()	consume result (no new output)	CompletableFuture<Void>	✗
thenRun()	run next step without input/output	CompletableFuture<Void>	✗
thenCompose()	flatten nested futures	CompletableFuture<R>	✗
thenCombine()	combine 2 futures	CompletableFuture<R>	✗
exceptionally()	handle exceptions	CompletableFuture<T>	✗
handle()	handle success/failure both	CompletableFuture<R>	✗
get()	wait & return result	T	✓
join()	like get, but unchecked exception	T	✓

---

8) thenApply vs thenApplyAsync (Important Interview Table)

Method	Runs where?	Uses executor?	Use case
thenApply()	Same thread used by previous stage	❌ default	lightweight transformations
thenApplyAsync()	potentially different thread (ForkJoinPool)	✅ default/commonPool	heavy CPU or independent stage

Example:

.thenApplyAsync(v -> v * 5)

---

9) Real-Time Scenario Example (Microservice Flow)

✅ Example: User places an order

Stage	CompletableFuture Step	Work
1	supplyAsync()	fetch user profile from DB
2	thenApply()	validate user
3	thenApply()	compute discount
4	thenApply()	return final bill

If you want, I can also provide a **complete CompletableFuture advanced table** including:
✅ thenCompose() vs thenCombine() with code
✅ exception handling patterns (production level)
✅ custom thread pool usage (Executor) with CompletableFuture

Below is the **complete practical answer in table form**: ✅ **When to use Multithreading** and ✅ **Executor Framework**, and ❌ **when NOT to use** them (real-time software situations).

✅ **When to Use vs When NOT to Use (Table)**

Scenario / Requirement	Use Multithreading?	Use Executor Framework?	Why / Real-time Explanation
Single CPU task, very small program	❌	❌	Thread creation + scheduling overhead is unnecessary
UI app responsiveness (Swing/JavaFX)	✅	✅	Keep UI thread free (background thread needed)
Multiple independent tasks (ex: 10 files processing)	✅	✅✅	Executor is best because it manages thread pool & task queue
CPU intensive parallel tasks (ex: image compression, hashing, ML inference)	✅✅	✅✅	Multithreading improves throughput; Executor gives better control
High volume tasks (1000+ tasks)	❌	✅✅✅	Creating 1000 threads will kill performance; thread pool needed
Backend server handling many users	❌	✅✅✅	Each request as task; pool ensures stability + scaling
Batch processing (ETL jobs)	✅	✅✅	Use executor to parallelize stages, control concurrency
Scheduled tasks (cron jobs, retry system)	❌	✅✅✅	Use ScheduledExecutorService for clean scheduling
I/O bound tasks (DB call, API call, file read)	✅	✅✅✅	Threads wait during I/O; executor manages efficiently

Massive I/O concurrency (10k+ connections)	✕	✓✓ (or Virtual Threads)	Classic pools can still become heavy → Virtual Threads best
Need return values from tasks	✕	✓✓✓	Use Callable + Future with executor
Need to cancel / timeout tasks	✕	✓✓✓	Executor supports cancellation & timeouts easily
Need ordering / task pipeline	✕	✓✓	Use executor queue / CompletableFuture
Simple one-time background task (1 task only)	✓	✓ (optional)	Thread can work; executor is better for consistency
Shared resource critical section (inventory update, booking system)	✓✓	✓✓	Use synchronization/locks + executor to control task execution
Highly latency-sensitive processing	□	✓✓	Thread pools reduce overhead, allow tuning
Need limited concurrency (ex: only 5 uploads at a time)	✕	✓✓✓	Fixed thread pool or semaphore-based executor pattern
Too much locking / synchronization already	✕	✓ (careful)	Threads may increase deadlocks & contention
Task must run sequentially (not parallel safe)	✕	✓ (single thread executor)	Use Executors.newSingleThreadExecutor()
Debugging in early prototype	✓	✓	Thread is easier to understand, executor is production style

Real-time trading / critical deterministic order	□	✅ (careful)	Concurrency introduces unpredictability; use only where safe
When tasks depend on each other heavily	❌	✅ (CompletableFuture)	Use async pipeline instead of raw threads
If you can do it using DB query / algorithm optimization	❌	❌	Multithreading is not a replacement for efficient logic

---

✅ Best Practice Decision Table (Quick Rule)

You need…	Best choice
1–2 small tasks only	Thread / Runnable
Many tasks (N tasks)	ExecutorService
Scheduling / recurring tasks	ScheduledExecutorService
Return values + cancellation	Callable + Future
Workflow chaining/pipeline	CompletableFuture
Maximum IO scalability	**Virtual Threads (Java 21)**
CPU parallelism	Fixed pool = number of cores / ForkJoinPool

❌ When You Should NOT Use Multithreading / Executor Framework

Situation	Why to avoid
Small simple work (fast loop, small logic)	Threads introduce overhead > benefits
Shared data heavily modified	Race condition risk high
Heavy locking required	Deadlocks + poor performance
Sequential business rule required	parallel execution breaks logic
Not thread-safe libraries used	may crash / corrupt data

You don't control thread count	may cause OutOfMemory / context switching
Work is actually single-core dependent	no speedup possible

✅ **Most Common Real-World Examples**

Use Case	What to use
Microservices request processing	Executor / Virtual Threads
File uploading system	Executor Fixed Pool
API calling 500 endpoints	Executor / Virtual Threads
Retry system + cron scheduler	ScheduledExecutorService
Bulk email sender	Executor + rate limiter
Data processing pipeline	CompletableFuture
Producer-consumer (Kafka consumer processing)	Executor + BlockingQueue

---

If you want, I can also provide:
✅ a **production-ready configuration guide** for thread pools (core size, max size, queue type, rejection policy)
✅ a **full Spring Boot example** showing executor pool usage in real microservices